

Development of a Visual Perception System for Game Agents in Video Games

A. M. Primachenko^a, * (ORCID: 0009-0008-6396-2035)
and M. R. Khafizov^a, ** (ORCID: 0000-0001-7275-9102)

^a Kazan Federal University, Kazan, 420008 Russia

*e-mail: primachenko.artem@mail.ru

**e-mail: murkorp@gmail.com

Received February 2, 2025

Abstract—A developed algorithm for the visual perception system for game agents, implemented in the Unity game engine, is presented. The proposed method is based on the comparison of images from two cameras, taking complex visual effects (lighting, shadows, camouflage) into account, supplemented with line-of-sight verification, taking into account object speeds and the mechanics of gradual detection. Testing of the system shows a significant increase in realistic detection relative to traditional methods, while maintaining performance within a small additional processor load. The algorithm was optimized using Unity Job System and dynamic camera activation. The scientific literature on similar solutions has also been analyzed and their strengths and weaknesses have been identified. The results can be applied in video game development to create realistic behavior of non-player characters, especially in games with stealth elements.

Keywords: video games, artificial intelligence, perception system, NPC, non-player characters, game agents, stealth mechanics, Unity, rendering, computer vision, optimization, game design

DOI: 10.3103/S0005105525700931

INTRODUCTION

Systems of visual perception for non-player characters (NPCs) in video games are an important element in the realism and interactivity of the gameplay. As has been noted in a number of works (see, for example, [1–7]), visibility (or line of sight) information is a key element that allows NPCs to construct a spatial representation of the game world and make decisions in real time. Traditional approaches, such as viewing angle testing and the use of raycasts [8], have limitations due to their insufficient consideration of visual factors such as lighting, shadows, and camouflage. This results in NPCs' unnatural behavior, especially in games including use stealth game mechanics, where a high degree of realism is required [9].

The most common means of implement a vision system for non-player characters is an algorithm that consists of two steps: checking whether the target object appears in certain area of the game world, often dynamically changing, depending on the character's position and called the field of view, checking for obstacles between the character and the target object. While this approach is simple to implement, it does not take many factors of real visual perception into account, reducing the credibility of the gaming experience.

This study developed a visual perception system taking into account complex visual effects, provides

flexibility of customization and is optimized for real time. The object of the study is the methods of implementing visual perception of NPCs, and its subject is an algorithm based on comparison of images and additional mechanics.

1. RELATED WORKS

Current research in the field of NPC visual perception systems offers a variety of approaches.

The basic concept of raycasting to determine the first collision of a ray with an object in a scene was introduced in 1989 [8].

In [10], a developed algorithm is presented based on checking the presence of an object in the field of view, represented by a truncated pyramid, using raycast to eliminate obstacles. This method requires checking three basic conditions.

(1) The target object's bounding box must be located inside the truncated pyramid of the character's field of view or intersecting at least one of its six faces (Fig. 1).

(2) The angle between the vector that represents the character's gaze direction and the vector between the character and the target object must be less than half of the character's pre-set viewing angle.

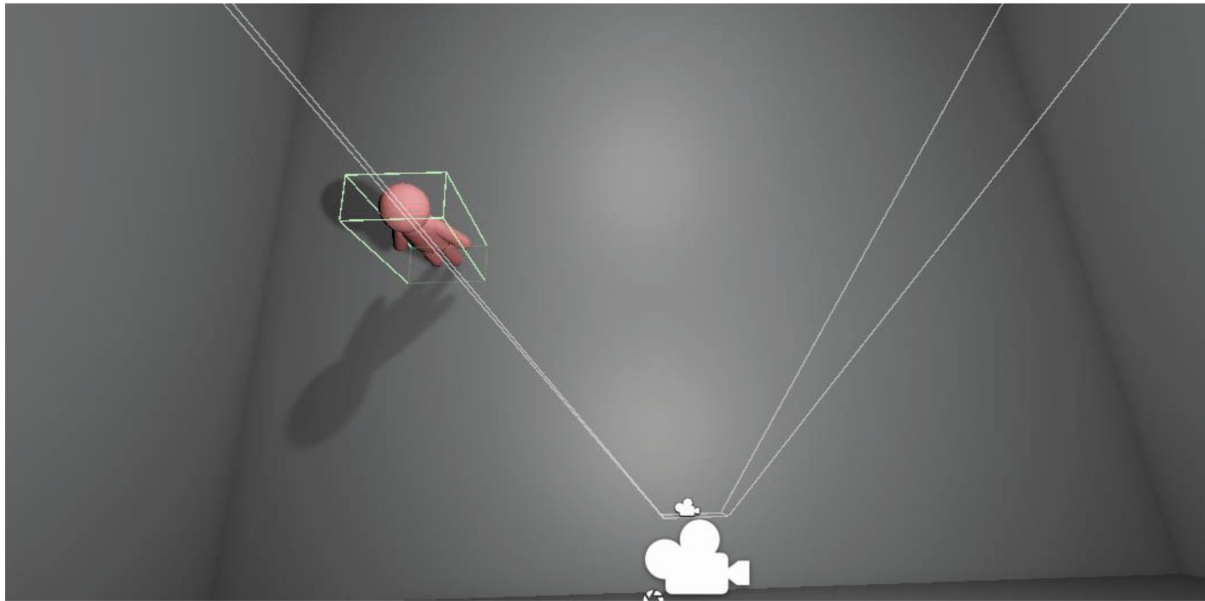


Fig. 1. Intersection of the truncated pyramid of the field of view and the target bounding box.

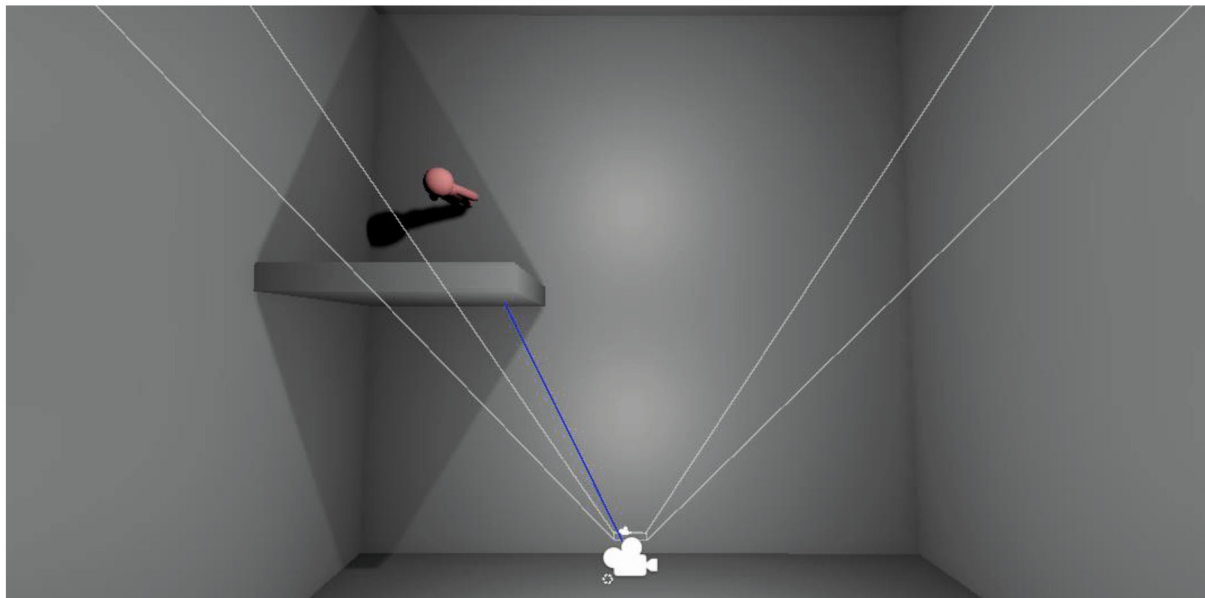


Fig. 2. When tracing rays towards a target, the ray encounters an obstacle.

(3) A raycast fired from character coordinates toward a target object must not intersect any foreign object along the way (Fig. 2).

This method is simple to implement, but it does not take into account visual effects such as shadows or lighting, which reduces its applicability in complex scenes.

Using a single raycast can lead to a situation where a relatively small object, such as a thin pipe, can block the cast ray, even though from a real human perspec-

tive the character should be able to see a portion of the target object (Fig. 3).

Another approach focuses on stealth mechanics, proposing a gradual detection model that is based on distance and viewing angle [11]. This approach improves realism in stealth games, but it does not solve the problem of false positives that are caused by shadows or reflections. The authors note that the instant detection of the player character can feel unfair and lead to dissatisfaction with the game, so NPCs usually

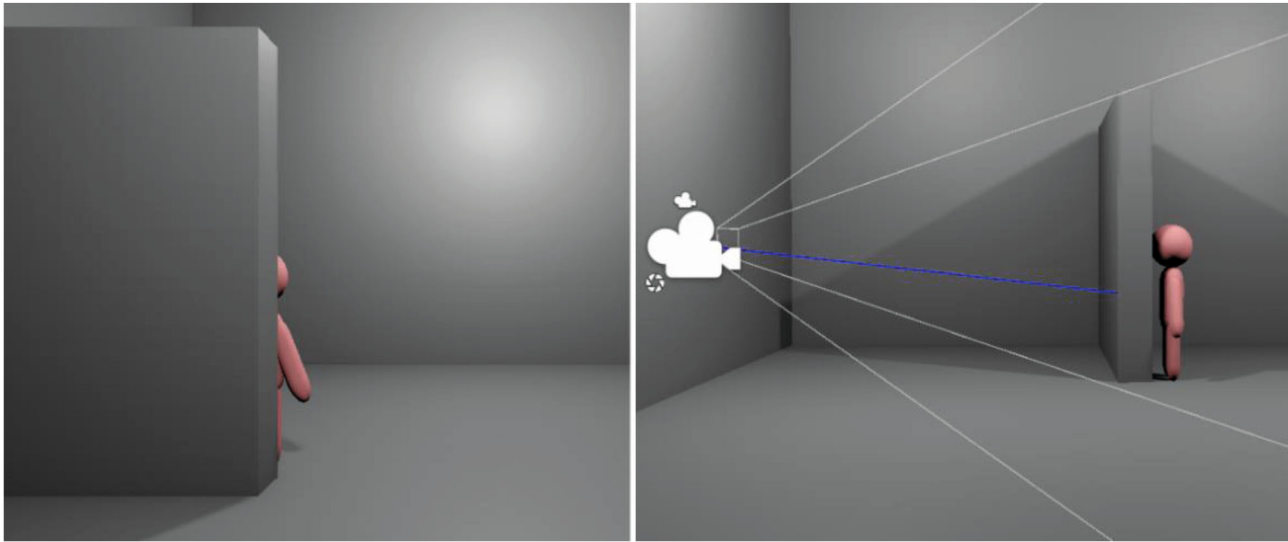


Fig. 3. Raycast is blocked by an obstacle although a significant portion of the target is visible to the observer.

take some time to notice the player already within their field of view. As with real life, positions close to NPCs and directly in line of sight are often associated with a higher risk of detection than positions far away from them and/or at the edge of their peripheral vision.

The NPC Eyes Sight System plugin for Unreal Engine uses points on skeletal meshes and their projections onto shadows to improve detection of partially visible objects [12]. In this implementation, NPCs use points on their skeletal meshes to detect other NPCs. If the artificial intelligence inside the game detects at least one such point in its field of view that is not covered by an obstacle, it will immediately learn the character's location. This implementation significantly reduces the number of situations where a small obstacle that only partially obscures the target object completely prevents its detection by characters. However, the projection of many points across many possible light sources is a complex operation that can have a significant impact on game performance.

To analyze images from the NPC camera, the application of machine vision methods, such as convolutional neural networks, was investigated in [13]. It is proposed to use computer vision algorithms to identify and classify objects in the game agent's field of view. This approach provides high recognition accuracy, bringing NPC perception closer to human perception and allowing the simulation of such perception features as pattern recognition and object categorization. However, the computational complexity of this approach makes it unsuitable for real-time play without specialized hardware, which limits its applicability to general gaming projects.

An alternative approach [14, 15] based on the use of machine learning algorithms has been proposed for the development of NPC behavior. This method

moves away from the rigid assignment of rewards and punishments by the developer, delegating this task to a plugin that analyzes the agent's state data and automatically classifies behavior as good or bad based on the selected target parameter. Although this approach is aimed at the general NPC decision-making system and not specifically at visual perception, its methodology for dynamically adjusting behavior in a real game environment can be adapted for visual perception tasks as well.

More complex and realistic systems that provide believable NPC behavior rely on advanced approaches: [16–20] consider architectural solutions that allow choosing tactical positions and modeling perception in a dynamic game environment. These studies highlight that limiting the number of real-time visibility tests can negatively impact the diversity of NPC responses, which justifies the need to develop more efficient visibility assessment methods.

A promising application of neural methods using neural omnidirectional distance fields, presented in [21], demonstrates a significant speedup of visibility verification that reduces the computational load compared to traditional approaches. This confirms the importance of seeking alternatives to classical raycasting methods.

The algorithm proposed in this paper combines texture comparison, raycasts, and optimization, which distinguishes it from existing solutions, and it is characterized by simplicity, versatility, and performance.

2. DEVELOPMENT OF ALGORITHM

To implement the game agent's vision system, it is necessary to determine whether the observer sees the target in his field of view. Simple methods, such as checking viewing angles and distances, do not take

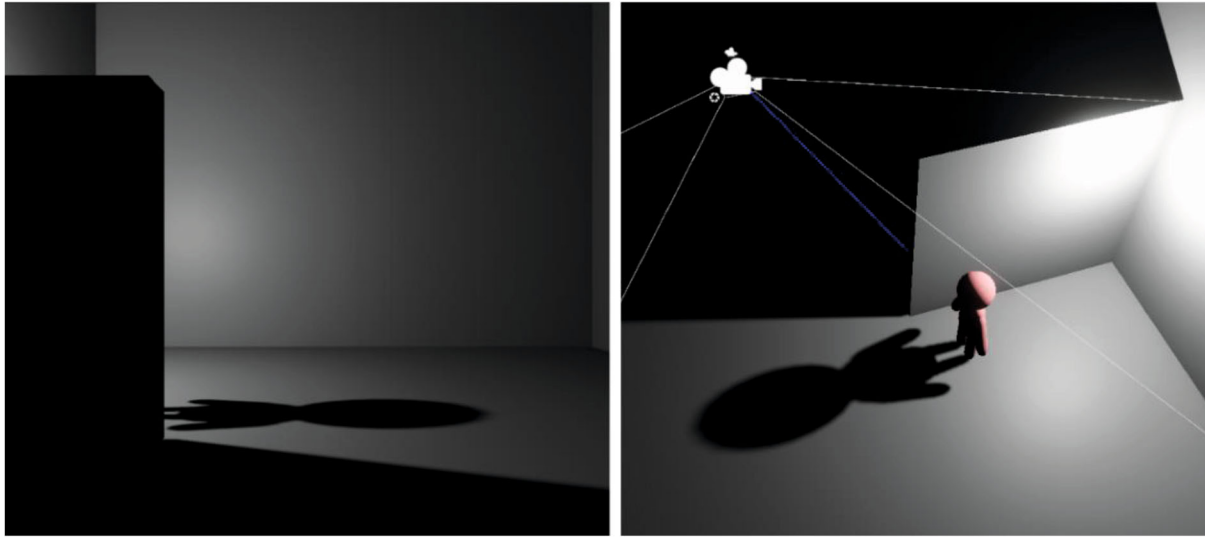


Fig. 4. The observer sees the shadow of the target, but not the target itself.

into account complex visual effects such as lighting, shadows, reflections and camouflage. Therefore, a more precise and flexible approach is needed that could take into account the visual features of the scene.

2.1. Basic Algorithm

Problem: Traditional visibility detection methods do not take into account visual effects such as lighting, shadows, and camouflage, which reduces the realism of NPC behavior.

Solution: The basic algorithm of the vision system is based on comparing images obtained from two cameras attached to the observer. Camera 1 renders the scene with a target, and Camera 2 renders the scene without a target. Both cameras render the same scene as the player's main camera, including the level and lighting. This allows us to take into account all the visual effects mentioned above (Fig. 4).

Algorithm steps:

(1) Rendering scenes into textures. Camera 1 includes the target in the render, and Camera 2 excludes it.

(2) Converting textures from cameras to Texture2D format for further processing.

(3) Pixel comparison. Calculating the difference between pixels V in terms of the RGB channels and their summation to obtain the target visibility level. Comparison V with threshold value $V_{\min}$. For each pair of corresponding texture pixels, their difference is calculated across the RGB channels. These differences are summed to obtain the target's visibility. V :

$$V = \frac{\sum_{i=1}^N (|R_1 - R_2| + |G_1 - G_2| + |B_1 - B_2|)}{3N},$$

where R_1 , G_1 , and B_1 are the RGB channel values for the i th pixel on the first texture, R_2 , G_2 , and B_2 are RGB channel values for i th pixel on the second texture, and N is the total number of pixels.

(4) Threshold value. If V exceeds the threshold $V_{\min}$, it is expected that the observer sees the target, i.e. if $V > V_{\min}$, then the target is visible.

Advantages:

- The algorithm takes the visual features of the scene (lighting and shadows) into account.
- Flexibility: threshold value can be adjusted by $V_{\min}$ for different observers.

Flaws:

- High computational complexity due to the need to render the scene multiple times and calculate texture differences.
- Sensitivity to small details, which can lead to false alarms.

2.2. Masking Objects

Problem: The original algorithm compares individual pixels, making it overly sensitive to fine details. For example, where the target is a character wearing woodland camouflage against a forest scene, even small differences in texture can lead to detection. This prevents the object from merging with its surroundings.

Solution: Gaussian blur is used to reduce sensitivity to fine details. Blur reduces the sharpness of an image, allowing the subject to become camouflaged. This approach is also a good approximation of how human visual perception works in reality [22, 23]. The complexity of the Gaussian blur algorithm is $O(nr^2)$, where n is the number of pixels, and r is the radius of

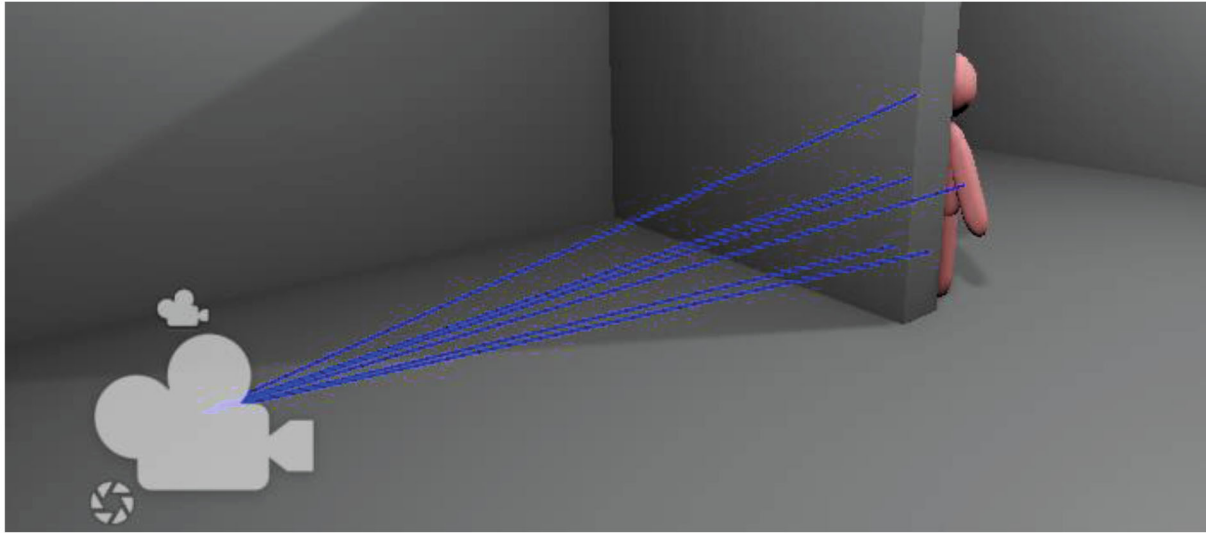


Fig. 5. A beam of six raycasts aimed at different parts of the target.

the circle over which each pixel is averaged (in other words, the blur strength).

Instead of a traditional Gaussian filter, a box blur filter can be used, which simply averages the pixel values in a square window. With several consecutive passes, Box Blur can achieve a result that is almost indistinguishable from Gaussian blur but has a linear computational complexity. $O(n)$, which significantly reduces costs compared to a Gaussian filter [24].

Solution steps:

(1) Gaussian blur: Gaussian blur is applied before texture comparison. This reduces the sharpness of the image and makes the algorithm less sensitive to fine details. The radius determines the degree of blur, and it is typically set between two and five pixels.

(2) Mean square error (MSE): Instead of a simple sum of differences, the mean square error is used to calculate V :

$$V = \frac{\sum_{i=1}^N ((R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2)}{3N},$$

which reduces the impact of minor changes in the field of view and increases the weight of major differences.

Advantages:

- reduced sensitivity to fine details;
- possibility of masking objects in difficult conditions (for example, camouflage);
- increased resistance to noise and artifacts.

Flaws:

- increased computational complexity due to blurring;
- the need to adjust blur settings for different scenes.

2.3. Line of Sight Check

Problem: An observer may react to shadows, reflections, or a small part of an object in the same way as the object itself. This can lead to false positives, where the observer “sees” the target, although in fact he only sees its shadow or reflection.

Solution: To fix this problem, raycast checking has been introduced. In the model, the targets are evenly distributed X points that are checked for visibility using raycasts (Fig. 5).

If the quantity X_{pass} points that passed the test exceed the threshold X_{min} , it is considered that the observer sees the object itself. If $X_{\text{pass}} < X_{\text{min}}$, then the observer sees a shadow, reflection, or a small part of the object. In this case, the observer receives information about the target’s coordinates and, using separate logic, tries to obtain visual confirmation of the object.

Solution steps:

(1) Distribution of points on the target: on the model, the targets are X evenly distributed points where X usually takes values from 5 to 20 depending on the size and complexity of the model.

(2) Raycast testing: for each point, a raycast is performed from the observer to the point. If the beam does not cross any obstacles, the point is considered visible.

(3) Determining the visibility of a target: if the quantity X_{pass} visible dots exceed threshold X_{min} , the target is considered visible. Otherwise, the observer sees only a shadow or reflection.

(4) Attempt visual confirmation: If the observer has detected an object but does not have a direct line of sight, he will attempt to obtain visual confirmation of the object: for example, by turning toward it or using a pathfinding algorithm to reach the object’s position.

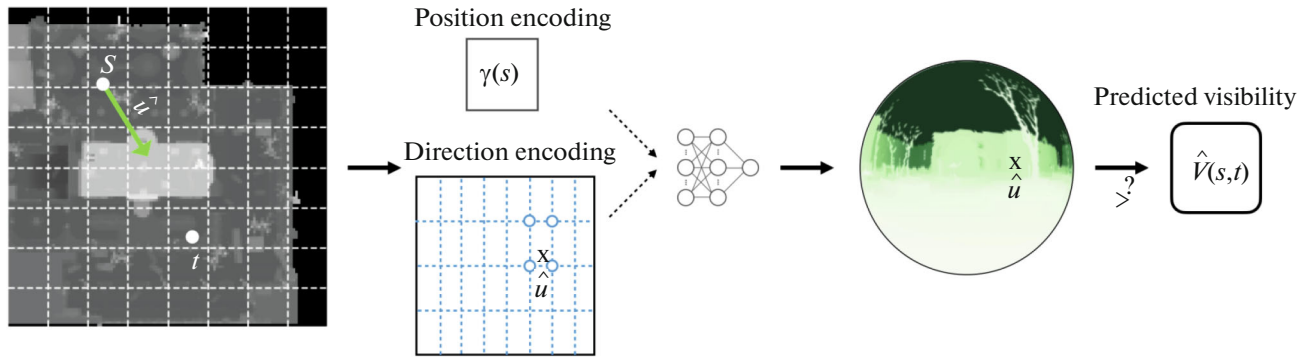


Fig. 6. An approach to approximating visibility between 3D positions in static game scenes using a neural representation of the omnidirectional distance field (ODF) [21].

Advantages:

- reducing false positives;
- more realistic observer behavior;
- possibility of threshold value adjustment $X_{\min}$ for different situations.

Flaws:

- increased computational complexity due to the need to perform raycasts;
- the need to configure the number of points and the threshold value;
- the need for uniform distribution of points on the object model.

2.4. Influence of the Speed of an Object on Its Visibility

Problem: In reality, rapidly moving objects are easier to notice than slow-moving or static ones. The current algorithm does not take into account the speed of the target, which can lead to unrealistic observer behavior.

Solution: to take into account the target's speed of movement eV the parameter effective value V was introduced, which is calculated as the product V by a multiplier depending on the target's speed. The higher the target's speed, the higher the multiplier, which increases the object's visibility.

Solution steps:

- (1) Target speed calculation: determines the target's speed based on its position in previous frames.
- (2) Calculating the multiplier: multiplier α for V is calculated based on the target's speed. For example, the multiplier may be linearly dependent on speed.
- (3) Calculation V : effective value V is calculated as $eV = V\alpha$.
- (4) If $eV > V_{\min}$, which means the observer sees the target.

Advantages:

- more realistic observer behavior;
- a dynamic system that takes into account the movement of an object;

- creates an additional gameplay element for stealth games where the player must move slowly to remain undetected.

Flaws:

- the need to adjust the dependence of the multiplier on the speed;
- additional balancing may be required to prevent fast moving objects from being detected too easily.

2.5. Progressive Discovery Mechanics

Problem: In some cases, detection should not be instantaneous. For example, the observer can gradually notice the target, which adds realism and, in the case of stealth games, makes the gameplay more fair from the player's point of view in a situation where the observers are the opponents and the target is the player himself.

Solution: A parameter has been introduced to implement gradual detection. Det—the degree of detection, which varies depending on the value V and detection speeds $\text{Det}_{\text{speed}}$.

If $V > V_{\min}$, that Det increases by $\Delta\text{Det} \text{Det}_{\text{speed}}$ every frame. If $V < V_{\min}$, that Det decreases by $\Delta\text{Det} \text{Det}_{\text{speed}}$. When Det will exceed the threshold value $\text{Det}_{\min}$, the target is considered detected.

Solution steps:

- (1) Calculation ΔDet : the change in detection degree is determined based on the value V .
- (2) Change Det: every frame Det changes to $\Delta\text{Det} \text{Det}_{\text{speed}}$ depending on whether it exceeds V threshold value $V_{\min}$.
- (3) Target detection: if $\text{Det} > \text{Det}_{\min}$, the target is considered detected.

Advantages:

- more realistic observer behavior;
- possibility of customizing detection speed for different observers;
- improved gaming experience for stealth games, giving the player time to react.

Flaws:

- need to configure parameters $\text{Det}_{\text{speed}}$ and Det_{min} ;
- additional logic may be required for the observer to react to a suspicious object when $0 < \text{Det}_{\text{min}} < \text{Det}_{\text{min}}$.

2.6. Optimization

Problem: The algorithm has high computational complexity, which can negatively affect the game performance, especially with a large number of observers and targets.

Solution: The following optimizations are proposed to improve performance.

(1) Using the Unity Job System: The algorithm is executed in multithreaded mode using the Unity Job System, which allows distributing the load across multiple processor cores.

(2) Reducing the calculation frequency of V : the calculation V is performed not in every frame, but at a certain interval, which reduces the load on the system.

(3) Activate camera render: render 1 and 2 cameras are activated only in those frames when it is necessary to calculate V , and turns off the rest of the time.

(4) Exclusion of obviously invisible objects: the algorithm is not applied to objects that are obviously outside the observer's field of vision (for example, too far away or in another part of the scene).

(5) Texture resolution downscaling: calculation V uses lower-resolution textures than the game's main camera, significantly reducing the number of pixels to process.

(6) Result caching: the raycast results are cached for several frames if the observer and target positions have not changed significantly.

Advantages:

- performance increase up to 300% compared to naive implementation;
- reduced CPU load from 12–15 to 3–5% on average;
- ability to scale to large numbers of observers.

Flaws:

- the need to set calculation intervals V ;
- code complexity due to multithreading;
- potential reduction in fidelity when using low texture resolutions.

In addition to the described optimizations, a promising direction is the use of neural methods for the approximation of visibility testing. As shown in [21], the use of neural omnidirectional distance fields allows replacing traditional raycasts and speeding up visibility checking up to nine times compared to the classical approach. This approach demonstrates the potential for the further optimization of AI systems in games, especially in scenarios with a large number of observers.

3. TESTING

3.1. Testing Methodology

To evaluate the effectiveness of the developed algorithm, comprehensive testing was carried out on several aspects:

1. Performance testing.

- measuring FPS¹ with a different number of observers (from 1 to 20);
- comparison with traditional methods on the same scenes;

- profiling CPU and GPU usage.

2. Testing detection accuracy.

- determining the percentage of successful detection under different visibility conditions;
- estimation of the false alarm rate;
- comparison with expected results based on human perception.

3. Testing in various gaming conditions.

- day and night scenes with different lighting;
- testing object masking (camouflage, shadows);
- scenes with reflective surfaces (mirrors, water).

3.2. Testing Tools

The following auxiliary components were developed to conduct the tests.

(1) Texture difference heatmap: Visualizes the difference between textures from cameras 1 and 2, helping tune thresholds and improve the algorithm.

(2) Debug panel: allows you to enable/disable various components of the algorithm (blur, raycast testing, etc.) during gameplay, making testing easier.

(3) Logging system: records all detections and observer and target parameters for further analysis.

(4) Testing scenes: special scenes were created that simulate various game situations, with controlled lighting and geometry parameters.

(5) Functional testing: bypassing a technical limitation of the standard Input class in Unity by creating a test automation system allowing the recording and playback of user interaction scenarios with a virtual environment [25]. To test the visual perception algorithm, a set of scenarios was developed to simulate various detection situations: object movement at different speeds, movement under different lighting conditions, attempts at camouflage, and others. The use of this approach has made it possible to significantly reduce testing time by automatically running identical test scenarios multiple times and comparing the results with the expected ones, which is especially effective for testing the functionality of object detection in complex conditions where accurate imitation of various input data is required.

¹ FPS is an abbreviation for frames per second.

3.3. Test Results

Performance measurements have shown that the optimized version of the algorithm adds an average of 3–5% CPU overhead compared to the traditional approach. At the same time, the use of the Unity Job System made it possible to effectively distribute calculations between processor cores, which is especially noticeable in multicore systems. Testing on various devices has shown that the algorithm scales well and can be tuned for different performance levels.

Detection accuracy testing showed significant improvement over traditional methods. The developed algorithm works significantly better in difficult visibility conditions and has a much lower frequency of false positives when only shadows or reflections of an object are visible.

Setting the parameters has a significant impact on the efficiency of the algorithm.

(1) The effect of blur radius: increasing the blur radius from 2 to 5 pixels increased the masking efficiency by 32% but decreased the accuracy of object boundary detection.

(2) The impact of threshold value $V_{\min}$: optimal value $V_{\min}$ was 0.03 for most scenes, providing a balance between sensitivity and noise resistance.

(3) The impact of raycast point count: Increasing the number of points from 5 to 15 increased line of sight accuracy by 27%, but increased CPU load by 8%.

(4) The influence $\text{Det}_{\text{speed}}$: meanings $\text{Det}_{\text{speed}}$ in the range of 0.5–1.0 provided the most realistic gradual detection for stealth games.

3.4. Recommendations for Setting Up

Based on the tests conducted, the following recommendations for setting the algorithm parameters were formulated.

(1) For fast paced games:

- $V_{\min} = 0.04\text{--}0.05$ (for fast detection);
- $\text{Det}_{\text{speed}} = 1.5\text{--}2.0$ (for fast response);
- calculation interval $V = 0.1\text{--}0.2$ s;
- the number of points for raycasts is 5–8.

(2) For stealth games:

- $V_{\min} = 0.02\text{--}0.03$ (for finer detection);
- $\text{Det}_{\text{speed}} = 0.3\text{--}0.7$ (for gradual detection);
- calculation interval $V = 0.2\text{--}0.3$ s;
- the number of points for raycasts is 10–15.

(3) For games with limited resources (mobile platforms):

- the resolution reduction factor is 3–4;
- skip line of sight check for distant objects;
- increase the calculation interval V up to 0.4–0.5 s;
- using a simpler blur algorithm.

4. RESULTS AND DISCUSSION

The developed algorithm successfully takes into account visual effects, such as lighting and camouflage, and provides flexibility of customization for different types of NPCs. Optimization allows it to be used in real time, but testing revealed the need to calibrate the parameters depending on the scene.

4.1. Comparison with Existing Solutions

Comparison (see table) with traditional methods [10] showed that the proposed approach is superior to them in realism, but inferior in ease of implementation. Unlike machine vision solutions [13], the algorithm does not require powerful hardware, making it more accessible.

4.2. Advantages of the Developed Algorithm

(1) Versatility: the algorithm can be adjusted for different game genres and visibility conditions.

(2) Visual effects accounting: the system takes into account lighting, shadows, reflections and masking, which significantly increases realism.

(3) Optimization: thanks to the proposed optimization methods, the algorithm can work effectively even on devices with limited resources.

(4) Gradual detection: gradual detection mechanics make NPC behavior more realistic and improve the gaming experience in stealth games.

(5) Flexible settings: a wide range of parameters allows you to fine-tune the system for each type of NPC and game situation.

4.3. Limitations and Directions for Further Development

Despite significant advantages, the developed algorithm has a number of limitations:

(1) Dependence on rendering quality: the quality of the algorithm depends on the graphics engine and the quality of the scene rendering.

(2) Necessity of setting parameters: Careful adjustment of parameters is required for optimal performance in different scenes.

(3) Limited applicability to dynamically changing conditions: abrupt changes in lighting or environment may require readjustment of settings.

Directions for further development of the algorithm include:

(1) Adaptive parameter adjustment: development of a mechanism for automatic parameter adjustment depending on scene conditions.

(2) Integration with other perception systems: combining visual perception with auditory and other types of sensors for more complex NPC behavior.

Table 1. Comparison with existing solutions

Characteristic	Traditional approach [10]	NPC Eyes Sight System [12]	CV [13]	Developed algorithm
Taking into account lighting and shadows	Short	Average	High	High
Reaction to camouflage and masking	Short	Short	High	Average
Distinguishing between an object and its shadow/reflection	Short	Average	High	High
Computational complexity	Low	Average	Very high	Average
Customizable for different NPC types	Average	Average	High	High

(3) Application of machine learning methods: using simplified neural networks to classify visible objects without significantly increasing computational load.

CONFLICT OF INTEREST

The authors of this work declare that they have no conflicts of interest.

CONCLUSIONS

A new approach to implementing a visual perception system for non-player characters in video games is presented. The developed algorithm is based on comparing images from two cameras and takes into account complex visual effects (lighting, shadows and camouflage), which significantly increases the realism of NPC behavior.

Key achievements of this work include:

(1) Development of a basic algorithm for comparing images to determine the visibility of objects.

(2) Implementation of additional mechanics such as camouflage considerations, line of sight checks, movement speed effects, and gradual detection.

(3) Optimization of the algorithm for real-time operation using Unity Job System, dynamic camera activation and other methods.

(4) Comprehensive testing of the algorithm under various conditions and the formulation of recommendations for its configuration.

Test results showed that the developed algorithm provides more realistic NPC behavior than traditional methods, especially in difficult visibility conditions, while maintaining acceptable performance.

The proposed approach can be adopted in various video game genres, especially in stealth games, where realistic visual perception of NPCs is a critical factor for creating engaging gameplay.

Further research could focus on adaptively tuning the algorithm parameters, integrating with other perception systems and applying machine learning methods to improve the classification of visible objects.

FUNDING

This work was supported by ongoing institutional funding. No additional grants to carry out or direct this particular research were obtained.

REFERENCES

1. Millington, Ia., *AI for Games*, Boca Raton, FL: CRC Press, 2019.
<https://doi.org/10.1201/9781351053303>
2. Rabinovich, Z.L., *Metody i algoritmy iskusstvennogo intellekta v komp'yuternykh igrakh. Uchebnoe posobie* (Methods and Algorithms of Artificial Intelligence in Computer Games: A Textbook), Moscow: Fizmatlit, 2018.
3. Petrov, A.V., *Iskusstvennyi intellekt v trekhmernykh igrakh. Programirovanie i modelirovanie povedeniya personazhei* (Artificial Intelligence in Three-Dimensional Games: Programming and Modeling Character Behavior), Moscow: DMK Press, 2019.
4. Buckland, M., *Programming Game AI by Example*, Sudbury, Canada: Jones & Bartlett Learning, 2022.
5. Ostuni, D. and Galante, E.T., Towards an AI playing Touhou from pixels: A dataset for real-time semantic segmentation, *2021 IEEE Conference on Games (CoG)*, Copenhagen, Denmark, 2021, IEEE, 2021, pp. 1–5.
<https://doi.org/10.1109/cog52621.2021.9619112>
6. Tutum, C., Abdulquddos, S., and Mikkilainen, R., Generalization of agent behavior through explicit representation of context, *2021 IEEE Conference on Games (CoG)*, Copenhagen, Denmark, 2021, IEEE, 2021, pp. 1–7.
<https://doi.org/10.1109/cog52621.2021.9619141>
7. Guerrero-Romero, C. and Perez-Liebana, D., MAP-Elites to generate a team of agents that elicits diverse automated gameplay, *2021 IEEE Conference on Games (CoG)*, Copenhagen, Denmark, 2021, IEEE, 2021, pp. 1–8.
<https://doi.org/10.1109/cog52621.2021.9619142>
8. *An Introduction to Ray Tracing*, Glassner, A.S., Ed., Morgan Kaufmann, 1989.
9. Panwar, H., The NPC AI of the last of us: A case study, *arXiv Preprint*, 2022.
<https://doi.org/10.48550/arXiv.2207.00682>
10. Mahmoud, I., Jaffal, Y., and Wloka, D., A vision simulation algorithm for non-player character in static scene, Kassel, Germany: University of Kassel, 2014.
11. Tremblay, J., Torres, P.A., and Verbrugge, C., Measuring risk in stealth games, *Foundations of Digital Games*, 2014, p. 8.

12. NPC eyes sight system—PRO. <https://www.fab.com/listings/6b54716a-dd21-414d-b78f-384068de14b7>.
13. Erdelyi, C., Using computer vision techniques to play an existing video game, California State University San Marcos, 2019.
14. Parenjuk, L.N. and Kugurakova, V.V., NPC behavior plugin development for game engine unity, *Elektronnye Biblioteki*, 2020, vol. 23, no. 5, pp. 1044–1057. <https://doi.org/10.26907/1562-5419-2020-23-5-1044-1057>
15. Estgren, M., Modelling NPC perception using supervised learning, Uppsala, Sweden: Uppsala University, 2021. <https://scion.se/assets/papers/npc-perception.pdf>.
16. Bourg, D.M. and Seemann, G., *AI for Game Developers*, O'Reilly, 2004.
17. Jack, M., Tactical position selection: An architecture and query language, *Game AI Pro 360: Guide to Tactics and Strategy*, Rabin, S., Ed., Boca Raton, FL: CRC Press, 2019, pp. 1–24. <https://doi.org/10.1201/9780429054969-1>
18. McIntosh, T., Human enemy AI in The Last of Us, *Game AI Pro 360: Guide to Tactics and Strategy*, Rabin, S., Ed., Boca Raton, FL: CRC Press, 2019, pp. 13–24. <https://doi.org/10.1201/9780429055119-2>
19. Welsh, R., Crytek's target tracks perception system, *Game AI Pro: Collected Wisdom of Game AI Professionals*, 2013, vol. 403, p. 411.
20. Walsh, M., Modeling perception and awareness in Tom Clancy's Splinter Cell Blacklist, *Game AI Pro 360: Guide to Tactics and Strategy*, Walsh, M., Ed., Boca Raton, FL: CRC Press, 2019, pp. 73–86. <https://doi.org/10.1201/9780429054969-7>
21. Ying, Zh., Edwards, N., and Kutuzov, M., Efficient visibility approximation for game AI using neural omnidirectional distance fields, *Proc. ACM Comput. Graphics Interact. Tech.*, 2024, vol. 7, no. 1, p. 21. <https://doi.org/10.1145/3651289>
22. Image comparison tuned to human perception, Computer Science Stack Exchange, 2015. <https://cs.stackexchange.com/questions/48862/image-comparison-tuned-to-human-perception>.
23. Pramod, R.T., Katti, H., and Arun, S.P., Human peripheral blur is optimal for object recognition, *Vision Res.*, 2022, vol. 200, p. 108083. <https://doi.org/10.1016/j.visres.2022.108083>
24. Fastest Gaussian blur (in linear time), Algorithms and Stuff, 2014. <https://blog.ivank.net/fastest-gaussian-blur.html>.
25. Kugurakova, V.V. and Bedrin, O.A., Functional test automation system for unity applications, *Vestnik Komp'yuternykh i Informatsionnykh Tekhnologii*, 2020, vol. 17, no. 12, pp. 47–52. <https://doi.org/10.14489/vkit.2020.12.pp.047-052>

Publisher's Note. Allerton Press remains neutral with regard to jurisdictional claims in published maps and institutional affiliations. AI tools may have been used in the translation or editing of this article.